Show What You Know: The Unofficial Guide

Colleges have two kinds of knowledge: the official kind (course catalogs, housing handbooks, university websites) and the real kind — the stuff students actually share with each other to survive. Rate My Professor reviews. Subreddit threads about which dining hall is worth the walk. Anonymous posts about which off-campus apartments have mold problems. Discord servers where seniors tell freshmen what professors actually want on exams.

In this project, you'll build **The Unofficial Guide**: a RAG (Retrieval-Augmented Generation) system that makes this kind of student-generated knowledge searchable and answerable. A user asks a plain-language question — "Is the housing lottery actually random?" or "Which CS professor gives the most useful feedback?" — and gets a grounded, cited answer drawn from real documents you collected.

This is your first production AI project. More structure is provided here than in later projects — use it to build the habits (spec first, evaluate honestly, document completely) that you'll need when that structure is gone.

🎯 Goals

By completing this project, you will be able to:

- Build an end-to-end document processing pipeline: ingestion, chunking, and embedding.
- Set up and query a vector store for semantic search.
- Generate grounded responses using retrieved context.
- Design and run an evaluation framework to measure how well your system actually works.
- Document your design decisions so someone else could understand and extend your system.

✅ Features

Required Features

Document Ingestion Pipeline: Collect and process at least 10 documents from your chosen domain. Your pipeline must: load the raw documents, clean or preprocess them as needed (remove navigation text, ads, etc.), and produce structured text ready for chunking. Describe this process in your README.
Chunking Strategy: Split your documents into chunks using a deliberate strategy — not just "split every 500 characters." Your `planning.md` must explain your chunk size, overlap, and why those choices fit your documents. For example, review-style text may warrant smaller chunks than long-form guides.
Vector Store and Semantic Search: Embed your chunks and store them in a vector database. Given a user query, retrieve the top relevant chunks using semantic similarity search. Your README should name the embedding model you used and reflect on what tradeoffs you'd consider if you were choosing for a production system (cost, context length, multilingual support, local vs. API).
Grounded Response Generation: Use an LLM to generate an answer to the user's query using only the retrieved chunks as context. Responses should not rely on the model's general knowledge — they should be grounded in what was retrieved. Include source attribution (which document(s) the answer draws from) in every response.
Query Interface: Build a basic interface for querying your system. This can be a simple web UI, a command-line tool, or a notebook — but it must be usable enough to demonstrate in your video without explaining how to navigate it.
Evaluation Report: Design 5 test questions with ground-truth answers, then run your system on each and evaluate the results. For each question, your report should document: the question, the correct answer, what your system returned, which chunks were retrieved, and whether the retrieval and response were accurate, partially accurate, or inaccurate. Identify at least one failure case and explain why it happened.

Stretch Features

Complete any of these for extra credit. Update your `planning.md` before starting each one.

Hybrid Search: Combine semantic search with keyword (BM25) search and compare results to semantic-only.

Chunking Strategy Comparison: Test 2+ chunking approaches on the same query set and report which performed better and why.
Metadata Filtering: Allow users to filter by document source, date, or rating (e.g., only show reviews from the past year).
Conversational Memory: Support multi-turn queries where the system remembers context from the previous question.

💡 Hints

- Collect your documents before you write any pipeline code. You'll make better chunking decisions once you've read what you're working with.
- Test retrieval before you add generation. A lot of RAG failures are retrieval failures — the LLM can't generate a good answer from bad chunks.
- Your evaluation should surface a failure. If all 5 test questions come back perfect, either your test questions are too easy or your evaluation criteria are too lenient. Make it harder.
- Source citations are not optional. A system that can't tell users where its answers came from isn't production-ready.
- If your system hallucinates (makes up something not in the documents), that's a valuable failure to document in your README — not something to hide.
- **If your documents are PDFs** (housing guides, syllabi, handbooks, etc.), use `pdfplumber` to extract text:

```
pip install pdfplumber
```

-

```
import pdfplumber
pdf = pdfplumber.open("file.pdf")
text = "\n\n".join(p.extract_text() for p in pdf.pages if p.extract_text())
```

-
- Note that `pdfplumber` does not perform OCR — scanned image-only PDFs will produce empty text. Digitally-created PDFs (anything you can select text in) work fine.

🛠️ Tools and Setup

This project uses a free tool stack — no paid subscriptions or API credits required.

Recommended stack

Component	Tool	Notes
Embeddings	`sentence-transformers` (`all-MiniLM-L6-v2`)	Runs locally — no API key, no rate limits
Vector store	ChromaDB	Runs locally — no account needed
LLM	Groq (`llama-3.3-70b-versatile`)	Free tier — sign up at console.groq.com

Getting started

Fork the Unofficial Guide starter repo, then **clone your fork** locally.
Create and activate a virtual environment from inside your cloned repo:

```
python -m venv .venv
source .venv/bin/activate          # Mac/Linux
source .venv/Scripts/activate      # Windows (Git Bash)
# or: .venv\Scripts\activate       # Windows (Command Prompt)
```

You should see `(.venv)` in your terminal prompt. Do this before installing anything — it keeps this project's dependencies isolated from the rest of your system.
Install dependencies — `requirements.txt` is already in the repo:

```
pip install -r requirements.txt
```

Set up your API key. Copy `.env.example` to `.env` in your repo root:

```
cp .env.example .env
```

Then replace `your_key_here` with your Groq API key. This file is already listed in `.gitignore` — never commit it. Get a free key at console.groq.com — no credit card required.

Milestone 1: Choose Your Domain and Collect Documents

⏰ **~30 min**

Before touching any code, decide what kind of student knowledge your system will make searchable and collect the raw material. Your documents are the foundation of everything — retrieval quality, chunking decisions, and evaluation design all depend on what you're actually working with. Read them before you build anything.

Choose one **domain** for your Unofficial Guide. A domain is a **topic or category of knowledge** — not a specific website. For example, "student reviews of CS professors at [university]" is a domain; Rate My Professors is a *source* you'd use to gather documents within that domain. Similarly, "off-campus housing experiences" is a domain; Reddit is a source. Keeping this distinction clear will help you stay focused when collecting documents.

Strong domain options: course and professor reviews, off-campus housing, campus dining, campus survival guides, or your own campus community. For each, sources might include Rate My Professors, department subreddits, housing forums, Yelp reviews, orientation wikis, or unofficial FAQs.

Identify at least 10 specific source documents, pages, or threads. Write down each source URL or file path. More sources means better coverage — aim for sources that together answer a range of different questions, not 10 pages that all say the same thing.

> Skim your documents before you do anything else. Notice how they're structured: Are they short reviews or long guides? Are the key facts concentrated in one sentence or spread across paragraphs? This will directly inform your chunking decisions in Milestone 2.
> Write a 2–3 sentence summary of your domain and what makes this knowledge hard to find otherwise. You'll use this in your `planning.md` and README.

📍 **Checkpoint**

You have at least 10 source documents identified (with URLs or file paths) and can describe in plain language what kinds of questions your system will be able to answer. If you can't describe 5 specific questions your system should handle, your domain may be too vague — narrow it down.

Make at least one commit before moving to Milestone 2.

Milestone 2: Write Your Spec Before Any Code

⏰ **~1 hour**

Write your `planning.md` before you write a single line of pipeline code. This isn't busywork — the decisions you make here shape every implementation choice downstream, and your spec is what you'll hand to an AI tool to generate code from. A clear spec produces useful AI-generated code. A vague one produces generic code that doesn't fit your system.

⚠️ **AI usage guardrail:** Do not ask your AI tool to fill in `planning.md` for you. Use it to understand concepts, pressure-test your decisions, and answer specific questions — not to generate the entire plan. A spec written by AI will produce a system you can't debug. Use the guiding questions and example prompts embedded in each section as starting points for those conversations.

> Open the `planning.md` already in your cloned repo. The section headers are pre-populated — fill them in with real content, not placeholders or "TBD."

`## Domain`

[What domain did you choose? Why is this knowledge valuable and hard to find through official channels?]

Documents
[List your specific sources: URLs, subreddit names, forum threads, or file descriptions. Aim for variety — sources that together cover different subtopics or perspectives within your domain.]

Chunking Strategy
[How will you split documents into chunks? State your chunk size (in tokens or characters), overlap size, and explain why those numbers fit the structure of your documents. A review-heavy corpus warrants different chunking than a long FAQ.

Guiding questions — use these to think it through before deciding:
- Are your documents short reviews (1–3 sentences) or long guides (many paragraphs)? How does that affect the right chunk size?
- If a key fact spans two adjacent chunks, will either chunk be retrievable on its own? What does overlap help with?
- How would you know if your chunks are too small? Too large? What would bad retrieval results look like in each case?

Useful AI prompts:
- "Explain how chunk size affects retrieval quality for short, opinion-based reviews."
- "What are the tradeoffs between chunking by paragraph vs. fixed character count for [my document type]?"
- "If I use 200-character chunks for review text, what kinds of queries might this fail for?"]

Retrieval Approach
[Which embedding model are you using (e.g., all-MiniLM-L6-v2 via sentence-transformers)? How many chunks will you retrieve per query (top-k)? If you were deploying this for real users and cost wasn't a constraint, what tradeoffs would you weigh in choosing a different embedding model — context length, multilingual support, accuracy on domain-specific text, latency?

Guiding questions:
- How many retrieved chunks is enough to give the LLM useful context? What happens if you retrieve too few? Too many?
- Why does semantic search find relevant chunks even when the query doesn't share exact words with the document?

Useful AI prompts:
- "What are different strategies for structuring embeddings for short, opinion-based text?"
- "What does top-k mean in a retrieval system, and what are the tradeoffs of setting it too high vs. too low?"]

Evaluation Plan

[List your 5 test questions with their expected correct answers. Questions should be specific enough that you can judge whether the system's response is right or wrong — "What are good dining halls?" is too vague; "What do students say about wait times at the [dining hall name] during lunch?" is testable.]

Anticipated Challenges
[What could go wrong? Consider: noisy or inconsistent documents, missing source attribution, off-topic retrieval, chunks that split key information across boundaries. Name at least two specific risks.]

AI Tool Plan
[Which parts of the pipeline do you plan to use AI tools (Claude, Copilot, ChatGPT, etc.) to help you implement? For each part, describe what you'll give the AI as input — which sections of this planning.md, which requirements from the instructions — and what you expect it to produce. Be specific: "I'll prompt Claude with my chunking strategy section and ask it to implement the chunk_text() function" is a plan. "I'll use AI to help me code" is not.]

> Draw a simple pipeline diagram and add it to your `planning.md` under a `## Architecture` header. It doesn't need to be polished — a hand-drawn sketch photographed and embedded, an ASCII diagram, or a Mermaid diagram are all fine. Your diagram should show the five stages: Document Ingestion → Chunking → Embedding + Vector Store → Retrieval → Generation. Label each stage with the tool or library you're using (e.g., "ChromaDB" on the vector store, "all-MiniLM-L6-v2" on the embedding step). You'll use this diagram as context when prompting AI tools to implement each stage.
> Review your evaluation plan questions — each one should have a specific, verifiable expected answer. If a question's "correct answer" is subjective, replace it with one that isn't.
> Update `planning.md` before starting any stretch features later.

📍 Checkpoint

`planning.md` contains all sections with substantive content, including an AI Tool Plan that names specific pipeline components you'll prompt AI to implement. Your pipeline diagram labels each stage with the tool you're using. Your evaluation plan includes 5 specific test questions, each with a clear expected answer that a grader could check a system response against.

Make at least one commit before moving to Milestone 3.

Milestone 3: Build the Document Pipeline

⏰ **~2–3 hours**

Your pipeline has two jobs: load your documents into memory and split them into chunks your embedding model can work with. Most RAG failures trace back to bad chunks — chunks that are too large dilute the relevant information, chunks that are too small lose context. Build this carefully and verify the output before moving on. Don't skip the chunk inspection step.

Start simple. If you haven't collected your documents yet, begin with plain `.txt` files rather than live web scraping. Some sources (like Rate My Professors) are difficult to scrape due to JavaScript rendering or blocked requests — you may need to copy text manually, use PDFs, or export to a structured format. This is normal and expected. Useful AI prompts if you hit scraping issues: "*What are different ways to extract text from a JavaScript-rendered website?*" or "*How can I convert unstructured text from a forum thread into a plain text file for processing?*"

The 2–3 hour estimate reflects a careful, incremental process — loading, cleaning, chunking, and validating at each step. If you find yourself finishing in 20 minutes by having AI generate everything at once, you're moving too fast. Come back and verify each stage before relying on it in the next.

> Use your `planning.md` as a prompt to an AI tool (Claude, Copilot, ChatGPT) to generate your ingestion and chunking code. Share your Documents section (what file types and sources you have), your Chunking Strategy section, and your pipeline diagram. Ask the AI to implement a script that loads your documents, cleans them, and produces chunks matching your specified chunk size and overlap. Review what it generates: does it match your spec? Does it handle the document structure you described? Correct anything that doesn't fit, and ask the AI to explain any part you don't understand.
> Write a script that loads all your documents. If you're scraping from URLs, collect the raw text. If you're using local files, load them from disk. Save the raw text to a consistent format before you start cleaning.

Clean each document. Remove anything that isn't the substantive content you want your system to use:

Remove: HTML tags, navigation menus, cookie banners, ads, footers, repeated site headers, "Read more" links, share buttons, comment counts, and any boilerplate that appears on every page.

Keep: The actual review text, opinions, ratings, descriptions, and any context needed to understand the content (e.g., the professor's name or course number in a review).

After cleaning, print one document and read it. If you still see nav text, leftover HTML entities (`&`, ` `), or content that doesn't belong to your domain, clean further before continuing.

Implement your chunking strategy from `planning.md`. Your implementation should use the chunk size and overlap you specified — if you're changing those numbers, update `planning.md` to reflect why.

Print 5 representative chunks and inspect them. For each, ask: does this make sense on its own? Could someone answer a question from this chunk alone, without reading what comes before or after?

Good chunk — a complete, retrievable thought:

"Professor Smith's exams are heavily based on lecture slides, not the textbook. Students consistently mention that attending every class is more important than doing the readings. Midterms are curved; finals are not."

Bad chunk (too small) — a fragment with no standalone meaning:

"Professor Smith's exams are heavily"

Bad chunk (too large) — multiple unrelated topics merged, too diluted to match any specific query:

[A 600-word chunk covering a professor's teaching style, their research interests, the department's advising policies, and unrelated comments about the building's parking situation]

Bad chunk (HTML artifact) — cleaning didn't finish:

`<div class="review-body">Professor Smith&#39;s exams are`

Count your total chunks and record the number. If you have fewer than 50 chunks across 10 documents, your chunks may be too large — each chunk is covering so much ground that specific queries can't match precisely. If you have more than 2,000, your chunks may be too small — each embedding carries so little meaning that the similarity search can't distinguish signal from noise.

📍 Checkpoint

Print 5 random chunks. Each one should be readable, substantive, and self-contained. If you see fragments, HTML, or empty strings, debug before embedding — bad chunks cannot be fixed by tuning retrieval later.

Common issues and how to diagnose them:

- **Empty chunks:** Your splitter is producing zero-length strings — add a `len(chunk) > 0` filter, or check if your documents loaded correctly.
- **HTML artifacts:** Cleaning didn't run or didn't catch all tags — print a raw document before cleaning and compare.
- **All chunks the same length:** Your chunker is splitting mechanically without respecting content boundaries — consider whether paragraph or sentence splitting fits your documents better.
- **Chunks from the wrong document:** Check that your metadata (source filename) is attached correctly to each chunk.

Make at least one commit before moving to Milestone 4.

Milestone 4: Embed Your Chunks and Test Retrieval

⏰ **~1-2 hours**

Embed your chunks and load them into a vector store, then test retrieval before you layer on generation. This step is where most retrieval bugs surface — and they're far easier to debug here than after you've wired in an LLM. Don't move to Milestone 5 until retrieval is returning relevant results.

> Use your `planning.md` Retrieval Approach section and your pipeline diagram to prompt an AI tool to generate your embedding and retrieval code. Give it your diagram to establish the full architecture, then ask it to implement the embedding step (loading chunks from your ingestion pipeline, embedding with `all-MiniLM-L6-v2`, storing in ChromaDB with source metadata) and a retrieval

function. If the generated code uses a ChromaDB API call or pattern you don't recognize, ask the AI to explain it — understanding what the code does is part of the exercise.

Set up your embedding model. The recommended default is `all-MiniLM-L6-v2` from `sentence-transformers` — it runs locally with no API key and no rate limits. Load it with `SentenceTransformer("all-MiniLM-L6-v2")`.

Embed all your chunks and load them into ChromaDB (or your chosen vector store) along with metadata for each chunk: at minimum, the source document name and the chunk's position in that document. You'll need source metadata later for attribution.

Write a retrieval function that accepts a query string and returns the top-k most relevant chunks along with their source information. Start with k=4 or k=5. If you retrieve too few chunks, the relevant content may not be in the set at all. If you retrieve too many, you dilute the context with loosely related material that can pull the LLM's response off-target. You'll tune this after you've seen real results.

Test retrieval with at least 3 of your 5 evaluation plan queries. For each, print the returned chunks and their distance scores. Ask: are these actually relevant to the question?

Good retrieval — specific, on-topic, from the right source:

Query: "What do students say about Professor Smith's exams?" Top result: "Professor Smith's midterms are heavily curved and focus on lecture slides. Multiple reviewers mentioned that attendance matters more than the textbook." (distance: 0.18)

Bad retrieval — wrong topic, or right topic but from the wrong source:

Top result: "The parking situation near the CS building has gotten worse since construction started." (distance: 0.61)*High distance score + off-topic content = retrieval failure, probably caused by chunks that are too small to carry enough semantic signal.*

If retrieval is returning chunks that seem unrelated, debug before moving on:

- **Print a retrieved chunk in full** — does it actually contain relevant content, or does it just have a few words in common with your query?
- **Check distance scores** — scores above 0.6–0.7 indicate weak matches. If your best result has a high score, your chunks may be too short or too noisy.
- **Check chunk content** — if chunks look like fragments or HTML leftovers, the cleaning/chunking stage didn't finish correctly.
- **Check metadata** — if results are coming from the wrong source, verify that each chunk was stored with the correct source filename in its metadata.
- **Adjust chunk size** — if retrieval consistently pulls loosely related content, try larger chunks that carry more semantic context per embedding.

📍 **Checkpoint**

Querying your vector store with 3 of your test questions returns chunks that visibly relate to each question. You can point to a returned chunk and explain why it's relevant to the query. Distance scores on your top results are below 0.5. If retrieval doesn't feel right, this is the time to fix it — generation won't compensate for poor retrieval.

Make at least one commit before moving to Milestone 5.

Milestone 5: Wire Up Generation and Build Your Interface

⏰ **~1-2 hours**

Connect retrieval to an LLM to generate grounded answers, then build a usable interface. The key engineering challenge here is grounding: your prompt must instruct the LLM to answer from the retrieved context only — not from its general training knowledge. Without this, your system will produce confident-sounding answers that have nothing to do with your documents.

> Use your `planning.md` and pipeline diagram to prompt an AI tool to generate the generation and interface code. Your prompt should include: your grounding requirement (answers from retrieved context only, with source attribution), the output format you want (answer + source list), and the Gradio skeleton structure if you're using it. Ask the AI to wire it all together. Before running the generated code, read through it — make sure the system prompt actually enforces grounding, not just suggests it, and that source attribution is programmatically guaranteed rather than left to the LLM to add on its own.
> Connect to your LLM. The recommended default is Groq's `llama-3.3-70b-versatile`, which is free-tier and OpenAI-compatible — initialize it with `from groq import Groq` and your `GROQ_API_KEY` from `.env`. Write a prompt template that passes the retrieved chunks as context and explicitly instructs the

model to answer only from that context. Example: "*Answer the question using only the information in the provided documents. If the documents don't contain enough information to answer, say 'I don't have enough information on that.*'"

Add source attribution to your response format. The LLM's response should name which document(s) the answer came from — either by instructing the model to cite sources in its response, or by appending retrieved source names programmatically after generation.

Test grounded generation end-to-end on 2–3 queries. The test: could this response have come from anywhere other than your retrieved chunks? If yes, it's a grounding failure — even if the answer happens to be correct.

Grounded response — answer traceable to retrieved text, source cited:

"According to student reviews of Professor Smith (source: rmp_smith_reviews.txt), exams are heavily curved and focus on lecture material rather than the textbook. Several reviewers specifically recommend attending every class."

Non-grounded response — draws on LLM training knowledge, no citation:

"Professor Smith likely structures exams similarly to most CS professors, emphasizing core concepts and problem-solving skills. It's generally a good idea to review lecture notes and practice past exams for any upper-division course."

The second response sounds authoritative and may even be correct — but it came from the model's training data, not your documents. If your system returns this kind of response, your grounding instruction needs tightening.

Also ask a question your documents don't cover. The system should explicitly say it doesn't have enough information — not generate a plausible-sounding answer from general knowledge.

Build your query interface. The recommended approach is a Gradio web UI — add `gradio>=6.9.0` to your `requirements.txt`, then `pip install gradio`. A minimal working interface looks like this:

```
import gradio as gr
from query import ask  # or wherever your end-to-end function lives

def handle_query(question):
    result = ask(question)
    sources = "\n".join(f"• {s}" for s in result["sources"])
    return result["answer"], sources

with gr.Blocks() as demo:
    inp = gr.Textbox(label="Your question")
    btn = gr.Button("Ask")
    answer = gr.Textbox(label="Answer", lines=8)
```

```
    sources = gr.Textbox(label="Retrieved from", lines=4)
    btn.click(handle_query, inputs=inp, outputs=[answer, sources])
    inp.submit(handle_query, inputs=inp, outputs=[answer, sources])

demo.launch()
```

> Run it with `python app.py` and open `http://localhost:7860`. You can also use Streamlit or a simple CLI — the requirement is that a viewer can understand how to use it from your demo video without narration explaining basic operation.

📍 **Checkpoint**

End-to-end: you enter a query, the system retrieves relevant chunks, and the response cites which document(s) it drew from. When you ask something your documents don't cover, the system declines to answer rather than generating something plausible but unfounded. Your interface is navigable without explanation.

Make at least one commit before moving to Milestone 6.

Milestone 6: Evaluate, Document, and Record

⏰ **~1.5–2 hours**

Run your evaluation plan, write your README, and record your demo. This is where your work becomes submittable — and where the hardest intellectual work happens. Identifying and honestly explaining a failure case is more valuable than having a system that appears to work perfectly. Graders are looking for evidence that you understand your system's limitations, not just that it runs.

> Run your system on all 5 test questions from `planning.md`. For each question, record in your README: the question, the expected answer, the system's actual response, and your accuracy judgment: **accurate**, **partially accurate**, or **inaccurate**.

Identify at least one failure case — a question where retrieval or generation didn't work as expected. Write a specific explanation of *why* it failed, tied to a part of the pipeline. "The answer was wrong" is not an explanation. "The relevant information was split across a chunk boundary, so the retrieval returned only half the context" or "The embedding model treated the professor's nickname as an out-of-vocabulary token and returned unrelated results" are explanations.
Complete your `README.md` using the template already in the starter repo. Every section has a guiding prompt — replace the prompts with your actual content. Every section is required; one-liners will not receive full credit.
Write your spec reflection in the README: describe one way the spec helped guide your implementation and one way your implementation diverged from it and why.
Add the AI usage section to your README. Describe at least 2 specific instances: what you asked the AI tool to do, what it produced, and what you changed, overrode, or directed differently.
Record a 3–5 minute demo video. Show: at least 3 different queries with source citations visible in the response, one query where retrieval and generation both work well, one query where the system struggles or fails (narrate what went wrong), and a brief walkthrough of your evaluation report.

📍 **Checkpoint**

All 5 evaluation questions are documented with accuracy judgments in your README. At least one failure is explained with a specific cause tied to the pipeline. README covers all required sections. Demo video is recorded and shows all required moments.

Make a final commit with your completed README and evaluation results before submitting.

📬 Submitting Your Project

Submit all of the following through the Course Portal:

Link to your forked GitHub repository

`planning.md` in your repo root (written before implementation, updated before stretch features)

`README.md` including:

- Domain and document sources (with specific names/URLs)
- Chunking strategy and reasoning (chunk size, overlap, why it fits your documents)
- Sample chunks: at least 5 labeled sample chunks, each with its source document name
- Embedding model used, and a brief reflection on what tradeoffs you'd consider when choosing a model for a production deployment
- Retrieval test results: at least 3 queries, each showing the query and the top returned chunks; for at least 2 of them, a written explanation of why the returned chunks are relevant
- How grounded generation is enforced in your system (prompt design or pipeline structure)
- Example responses: at least 2 responses with source attribution visible in the output text, plus one out-of-scope query showing the system's refusal response
- Query interface: a description of the input and output fields, and a sample interaction transcript showing one complete query and response
- Evaluation report: all 5 test questions with expected answers, system responses, and accuracy judgments
- At least one honest failure case with a specific explanation of why it happened
- Spec reflection: one way the spec helped you, one way implementation diverged from it and why
- AI usage section: at least 2 specific instances describing what you directed the AI to do and what you revised or overrode

Demo video (3–5 minutes) showing:

- At least 3 different queries answered with source citations visible in the response
- One query where retrieval works well (briefly narrate why the retrieved chunks are relevant)
- One query where the system struggles or fails (narrate what went wrong)
- A walkthrough of the evaluation report